

Efficient Memory Management for Large Language Model Serving with PagedAttention

📅 Date	May 12, 2026
☰ Multi-select	#NLP



arxiv.org

<https://arxiv.org/pdf/2309.06180>

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work 관련 연구

3. 제안 방법론

4. 실험 및 결과

5. 결론 (배운점)

1. Introduction

논문이 다루는 분야

대규모 언어 모델(LLM) 서빙에서의 고처리량(high throughput) 및 효율적인 메모리 관리

1. LLM 서비스의 부상과 경제적 도전 과제

- GPT, PaLM과 같은 대규모 언어 모델(LLM)의 등장은 프로그래밍 어시스턴트나 챗봇 등 혁신적인 애플리케이션을 가능하게 함. 많은 클라우드 기업들이 이를 서비스화하기 위해 경쟁하고 있으나, LLM 요청 처리 비용은 기존 키워드 검색 대비 약 10배 더 비쌈

- 높은 운영 비용 때문에 LLM 서빙 시스템에서 처리량을 높여 요청당 비용을 절감하는 것이 매우 중요한 과제가 됨

2. LLM 서빙의 기술적 병목: 메모리 바운드(Memory-bound)

- **자기회귀적 생성(Autoregressive Generation):** LLM은 토큰을 하나씩 순차적으로 생성하며, 각 단계마다 이전 토큰들의 Key-Value 벡터인 KV 캐시(KV Cache)를 참조함
- **메모리 효율성 문제:** 이는 계산 능력보다 메모리 대역폭이나 용량에 의해 성능이 제한되는 메모리 바운드 작업. 처리량을 높이기 위해 여러 요청을 배치 처리해야 하지만, 각 요청이 점유하는 KV 캐시의 크기가 매우 크고 동적으로 변하기 때문에 메모리 관리가 어려움
 - **메모리 점유 예시:** NVIDIA A100(40GB)에서 13B 모델을 서빙할 때, 모델 가중치는 약 65%를 차지하고 KV 캐시는 약 30%를 점유. 이 KV 캐시를 얼마나 효율적으로 관리하느냐가 최대 배치 크기와 시스템 처리량을 결정짓는 핵심 요소

해당 task에서 기존 연구 한계점

기존의 LLM 서빙 시스템(FasterTransformer, Orca 등)은 다음과 같은 이유로 메모리를 비효율적으로 관리함

- **연속 메모리 할당의 강제:** 대부분의 딥러닝 프레임워크는 텐서를 연속적인 메모리 공간에 저장하도록 요구함 → 기존 시스템은 요청의 최대 시퀀스 길이(예: 2048 토큰)에 맞춰 미리 연속된 메모리 덩어리를 할당함
- **메모리 단편화**
 - 내부 단편화(Internal fragmentation): 실제 생성된 문장이 할당된 최대 길이보다 훨씬 짧을 때 발생하는 낭비
 - 예약 낭비(Reservation waste): 미래에 생성될 토큰을 위해 미리 공간을 비워두기 때문에, 다른 요청이 이 공간을 활용X
 - 외부 단편화(External fragmentation): 요청마다 할당된 크기가 달라 메모리 할당기 자체에서 낭비
- **메모리 활용률 저조:** 조사 결과, 기존 시스템에서 실제 토큰 상태 저장에 사용되는 유효 메모리는 전체의 20.4% ~ 38.2%에 불과했음
- 병렬 샘플링이나 빔 서치 같은 고급 디코딩 알고리즘은 여러 시퀀스가 KV 캐시의 일부를 공유할 수 있는 기회가 많지만, 기존의 연속 저장 방식으로는 이러한 메모리 공유를 구현하기가 매우 어려움

논문의 contributions

- **PagedAttention 제안**

운영체제의 가상 메모리와 페이징(Paging) 기법에서 영감을 얻어, KV 캐시를 물리적으로 연속되지 않은 블록 단위로 저장하고 관리하는 새로운 어텐션 알고리즘을 제안

- **vLLM 시스템 구축**

PagedAttention을 기반으로 한 고성능 분산 서빙 엔진인 **vLLM**을 설계하고 구현함. vLLM은 블록 단위의 메모리 관리와 선점형 스케줄링을 통해 메모리 낭비를 거의 0(near-zero)에 가깝게 줄임

- 빔 서치나 병렬 샘플링 시 시퀀스 간에 KV 캐시 블록을 유연하게 공유할 수 있게 하여 메모리 사용량을 추가로 절감함
- 다양한 모델(GPT, OPT, LLaMA)에 대한 실험을 통해, 기존 최신 시스템인 FasterTransformer 및 Orca 대비 처리량을 2~4배 향상시킬 수 있음을 입증. 이 성과는 긴 시퀀스나 복잡한 디코딩 알고리즘을 사용할 때 더욱 효과적임

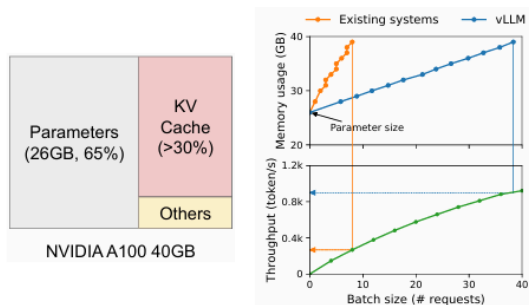


Figure 1. Left: Memory layout when serving an LLM with 13B parameters on NVIDIA A100. The parameters (gray) persist in GPU memory throughout serving. The memory for the KV cache (red) is (de)allocated per serving request. A small amount of memory (yellow) is used ephemerally for activation. Right: vLLM smooths out the rapid growth curve of KV cache memory seen in existing systems [31, 60], leading to a notable boost in serving throughput.

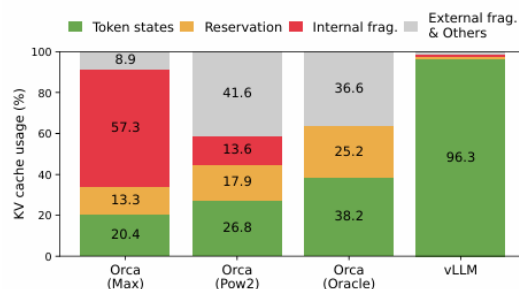


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

2. Related Work 관련 연구

- **일반 모델 서빙 시스템:** Clipper, TensorFlow Serving 등 기존 시스템은 배치 처리나 스케줄링을 연구했으나, LLM의 자기회귀(autoregressive) 특성과 토큰 상태(KV 캐시)를 고려하지 않아 최적화 기회에 한계가 있음

- **트랜스포머 전용 서빙 시스템:** FasterTransformer는 지연 시간 최적화에 집중했으나 효율적인 스케줄링이 부족하며, Orca는 반복 수준 스케줄링을 도입했으나 여전히 연속 메모리 할당 방식의 한계를 가짐
- **메모리 최적화 기법:** FlexGen은 오프라인 설정을 대상으로 가중치와 상태를 스와핑하는 방식을 연구했으나 온라인 서빙에는 적합하지 않았으며, FlashAttention은 계산 중 최고 메모리 사용량을 줄이는 데 집중함

3. 제안 방법론

- **PagedAttention:** KV 캐시를 고정된 크기의 KV 블록으로 나눔. 이 블록들은 운영체제의 페이지처럼 물리 메모리상에서 불연속적으로 배치될 수 있음
- 운영체제의 가상 메모리 관리와 유사하게 논리적 블록을 물리적 블록으로 매핑하는 블록 테이블(Block Table)을 관리함
 - 새로운 토큰이 생성될 때마다 필요한 물리 블록을 동적으로 할당하여 메모리 낭비 방지
- **메모리 공유 (Copy-on-Write):** 여러 시퀀스가 동일한 프롬프트를 공유할 때 동일한 물리 블록을 가리키게 하며, 변경이 필요한 시점에만 블록을 복사하는 **Copy-on-Write(COW)** 기법을 통해 빔 서치 등의 복잡한 알고리즘에서 메모리 사용량을 획기적으로 줄임
- 메모리가 부족할 경우 **FCFS(First-Come-First-Serve)** 정책에 따라 요청을 선점 (preemption)하며, 이때 KV 캐시를 CPU 메모리로 옮기는 **스와핑(Swapping)** 또는 다시 계산하는 **재계산(Recomputation)** 방식 사용

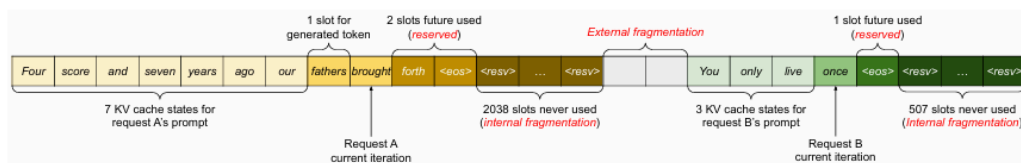


Figure 3. KV cache memory management in existing systems. Three types of memory wastes – reserved, internal fragmentation, and external fragmentation – exist that prevent other requests from fitting into the memory. The token in each memory slot represents its KV cache. Note the same tokens can have different KV cache when at different positions.

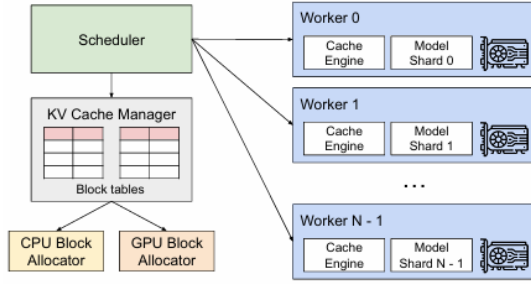


Figure 4. vLLM system overview.

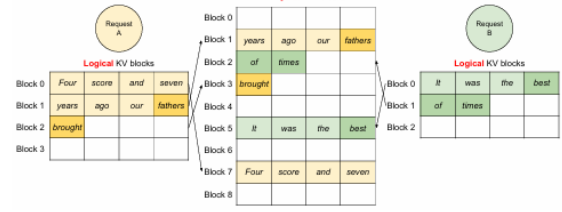


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

4. 실험 및 결과

- vLLM은 동일한 지연 시간 수준에서 기존 최신 시스템(FasterTransformer, Orca) 대비 2~4배의 처리량 개선 달성함
- 기존 시스템이 실제 토큰 상태 저장에 메모리의 20.4~38.2%만 사용한 반면, vLLM은 약 96.3%의 높은 메모리 활용률을 보임
- 복잡한 디코딩 최적화- 빔 서치에서 Orca 대비 최대 2.3배, 프롬프트 공유 시나리오에서 최대 3.58배 더 높은 처리량을 기록함
- PagedAttention 도입으로 인한 커널 자체의 오버헤드는 FasterTransformer 대비 약 20~26% 정도이나, 이는 전체 모델 실행 시간에서 작은 비중을 차지하므로 최종적인 서버 처리량은 훨씬 높음

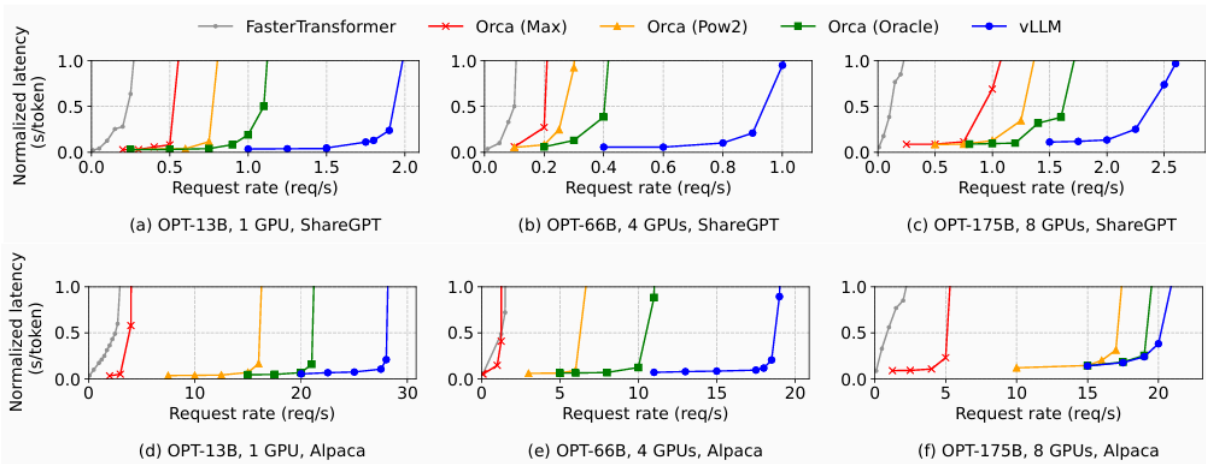


Figure 12. Single sequence generation with OPT models on the ShareGPT and Alpaca dataset

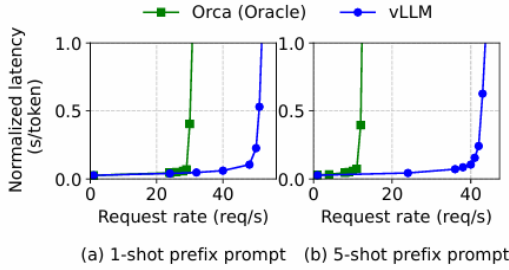


Figure 16. Translation workload where the input prompts share a common prefix. The prefix includes (a) 1 example with 80 tokens or (b) 5 examples with 341 tokens.

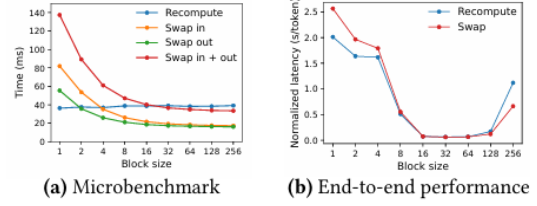


Figure 19. (a) Overhead of recomputation and swapping for different block sizes. (b) Performance when serving OPT-13B with the ShareGPT traces at the same request rate.

5. 결론 (배운점)

- 운영체제의 핵심 개념인 가상 메모리와 페이징 기법을 현대적인 LLM 서빙 인프라의 메모리 병목 문제에 성공적으로 이식할 수 있음을 보여줌
- 계산 능력보다 메모리 용량이 병목이 되는 LLM 서빙 환경에서는 단편화를 제거하고 메모리를 유연하게 공유하는 것이 처리량을 극대화하는 가장 효과적인 방법임을 보임
- 단순한 페이징을 넘어, 모든 블록이 함께 접근된다는 특성을 고려한 '전부 혹은 전무(all-or-nothing)' 스왑 정책이나 재계산 기법 등 LLM 특화 최적화가 중요함을 배움